CS471 – Web Technologies (Laboratory)	Qassim University College of Computer	Lab Week 7
		Django Models (Part 1)

This lab session covers the basics of Django Models (Part 1). By the end of this session, students should have a very good understanding of how to build a basic model and how to access models in Django.

Pre-lab Preparation:

1. Having bookmodule and usermodule in apps directory, with necessary configurations in settings.py.

Lab Activities:

Task 1: using models, we create a database scheme

ID	Title	Authors	Price	Edition
1	Continuous Delivery	J.Humble and D. Farley	120.00	3
2	Reversing: Secrets of Reverse Engineer	E. Eilam	97.00	2
3	The Hundred-Page Machine Learning Book	Andriy Burkov	100.00	4

Step 1: To have the scheme in the database, we write the following in 'apps/ bookmodule/models.py', Note that: ID is done automatically by Django, so we do not have to explicitly define it in the model.

```
from django.db import models

class Book(models.Model):
    title = models.CharField(max_length = 50)
    author = models.CharField(max_length = 50)
    price = models.FloatField(default = 0.0)
    edition = models.SmallIntegerField(default = 1)
```

models.py

C: > Users > Lenovo > djangoenv > DjangoProjects > libraryproject > apps > bookmodule > mode

```python
from django.db import models

# Create your models here.

from django.db import models

class Book(models.Model):
    title = models.CharField(max_length=50)
    author = models.CharField(max_length=50)
    price = models.FloatField(default=0.0)
    edition = models.SmallIntegerField(default=1)
```

Step 2: Using manage.py from the command line, to make necessary migrations (modification that the database need to do):

> python manage.py makemigrations

Step 3: Again using manage.py from the command line, apply the migrations to the database:

> python manage.py migrate

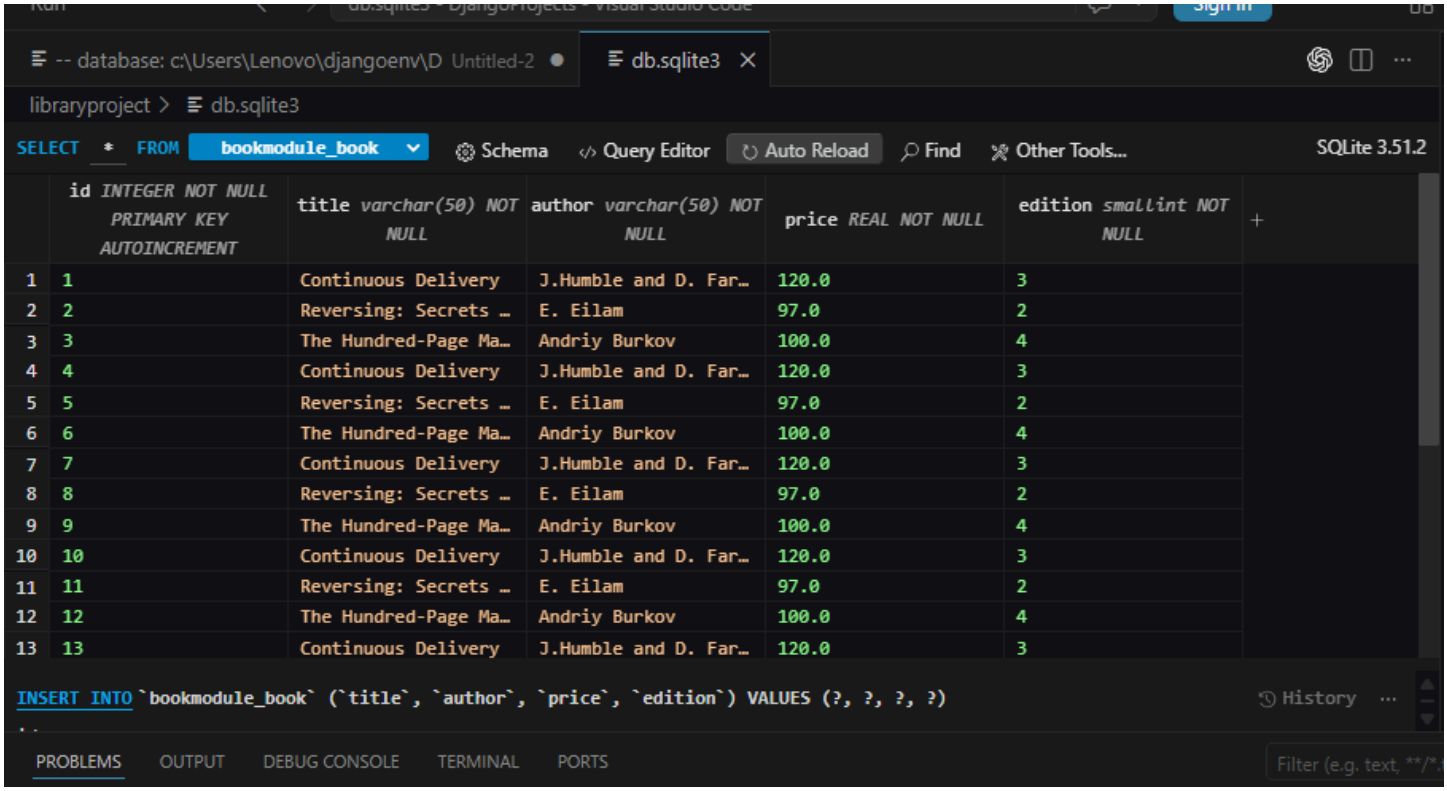
```
C:\Users\Lenovo\djangoenv\DjangoProjects\libraryproject>C:\Users\Lenovo\djangoenv\Scripts\activate

(djangoenv) C:\Users\Lenovo\djangoenv\DjangoProjects\libraryproject>python manage.py makemigrations
Migrations for 'bookmodule':
  apps\bookmodule\migrations\0001_initial.py
    + Create model Book

(djangoenv) C:\Users\Lenovo\djangoenv\DjangoProjects\libraryproject>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, bookmodule, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
```

Task 2: use the application 'DB Browser for SQLite' or 'VS code' to view the database and insert fake data into it.

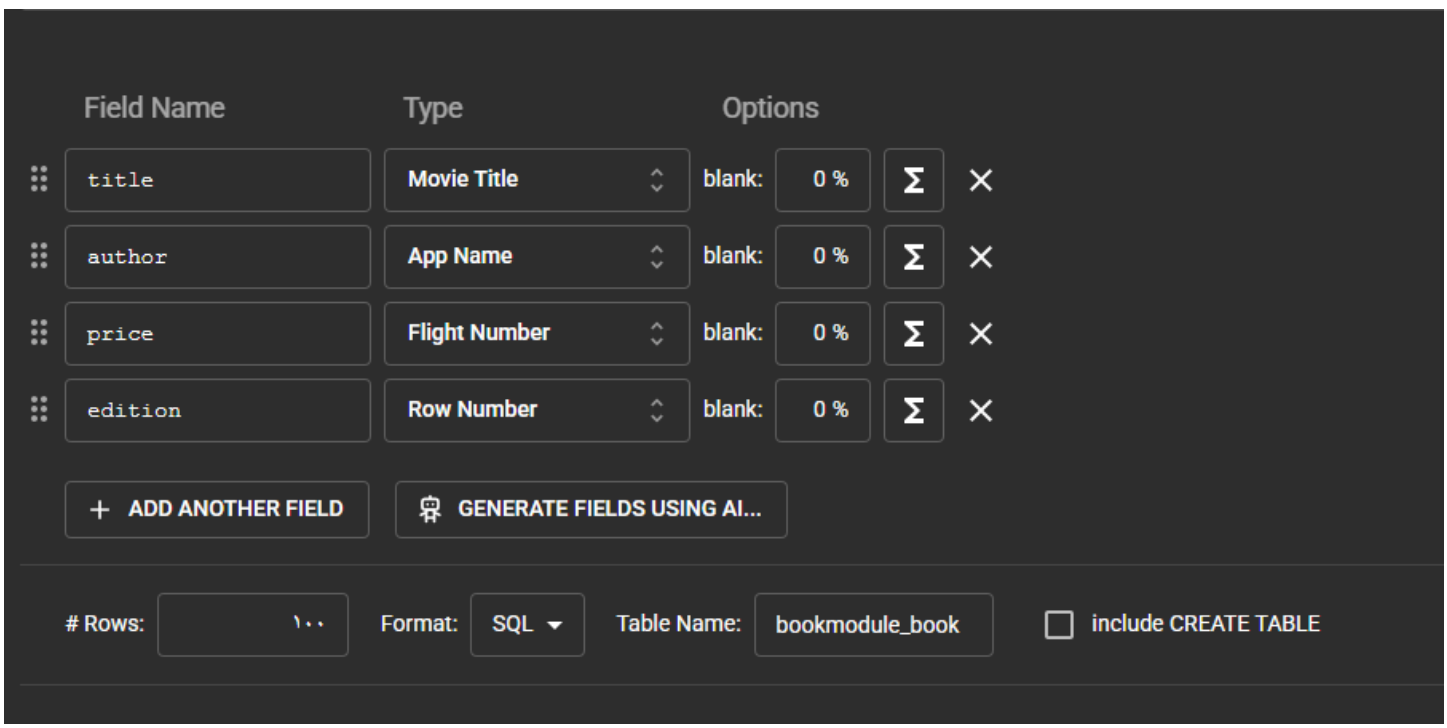
Note: For VS code, you need *SQLite extension*, and for SQLite, you will use *sqlitebrowser* at (https://sqlitebrowser.org/dl/). In the lab, we assumed that you are familiar with at least one of them.

Step 1: Open the database 'db.sqlite3' file in 'VS code' or 'DB Browse for SQLite' apps you can see the table.

Step 2: For development purposes, you would need data, and you can use random data generator websites, like: www.mockaroo.com



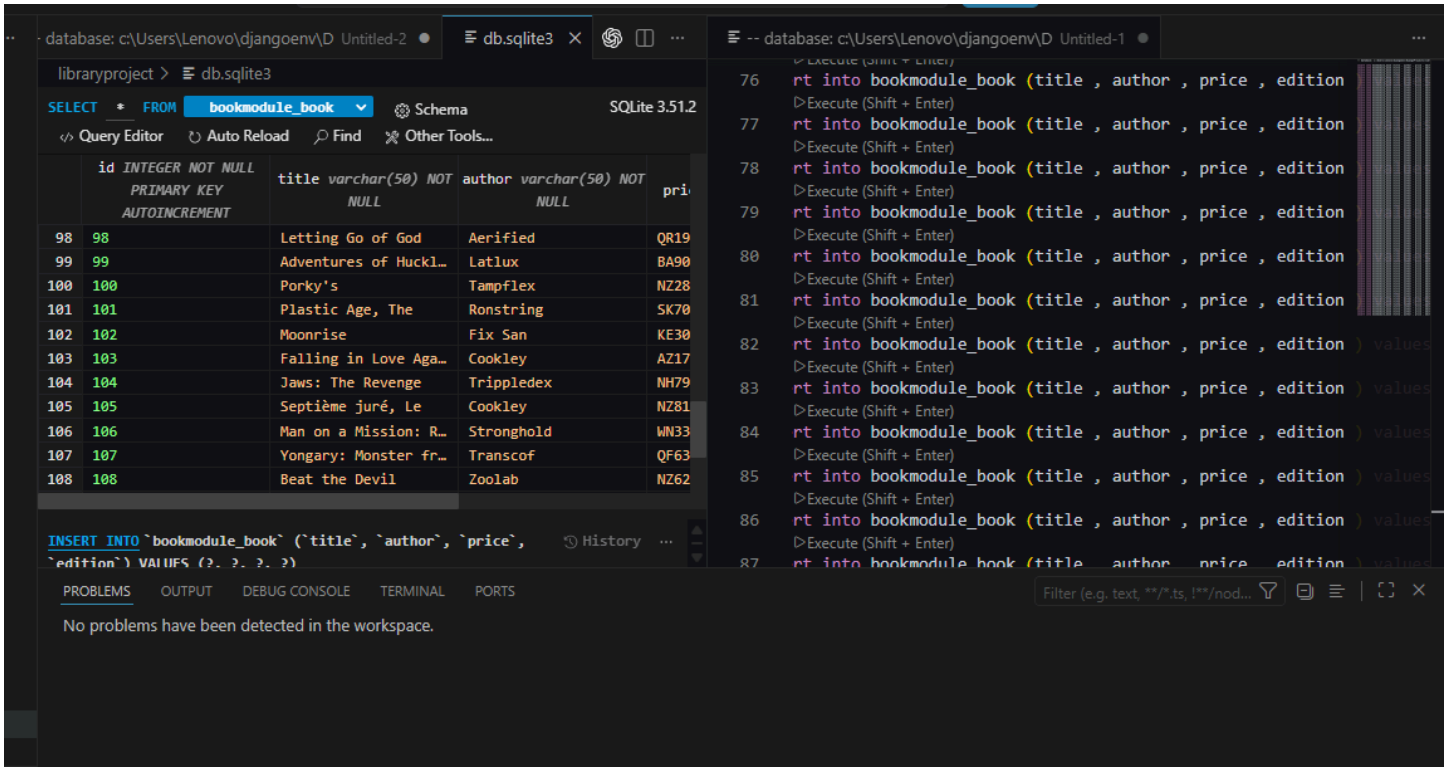
	id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT	title varchar(50) NOT NULL	author varchar(50) NOT NULL	price REAL NOT NULL	edition smallint NOT NULL
1	1	Continuous Delivery	J.Humble and D. Far…	120.0	3
2	2	Reversing: Secrets …	E. Eilam	97.0	2
3	3	The Hundred-Page Ma…	Andriy Burkov	100.0	4
4	4	Continuous Delivery	J.Humble and D. Far…	120.0	3
5	5	Reversing: Secrets …	E. Eilam	97.0	2
6	6	The Hundred-Page Ma…	Andriy Burkov	100.0	4
7	7	Continuous Delivery	J.Humble and D. Far…	120.0	3
8	8	Reversing: Secrets …	E. Eilam	97.0	2
9	9	The Hundred-Page Ma…	Andriy Burkov	100.0	4
10	10	Continuous Delivery	J.Humble and D. Far…	120.0	3
11	11	Reversing: Secrets …	E. Eilam	97.0	2
12	12	The Hundred-Page Ma…	Andriy Burkov	100.0	4
13	13	Continuous Delivery	J.Humble and D. Far…	120.0	3

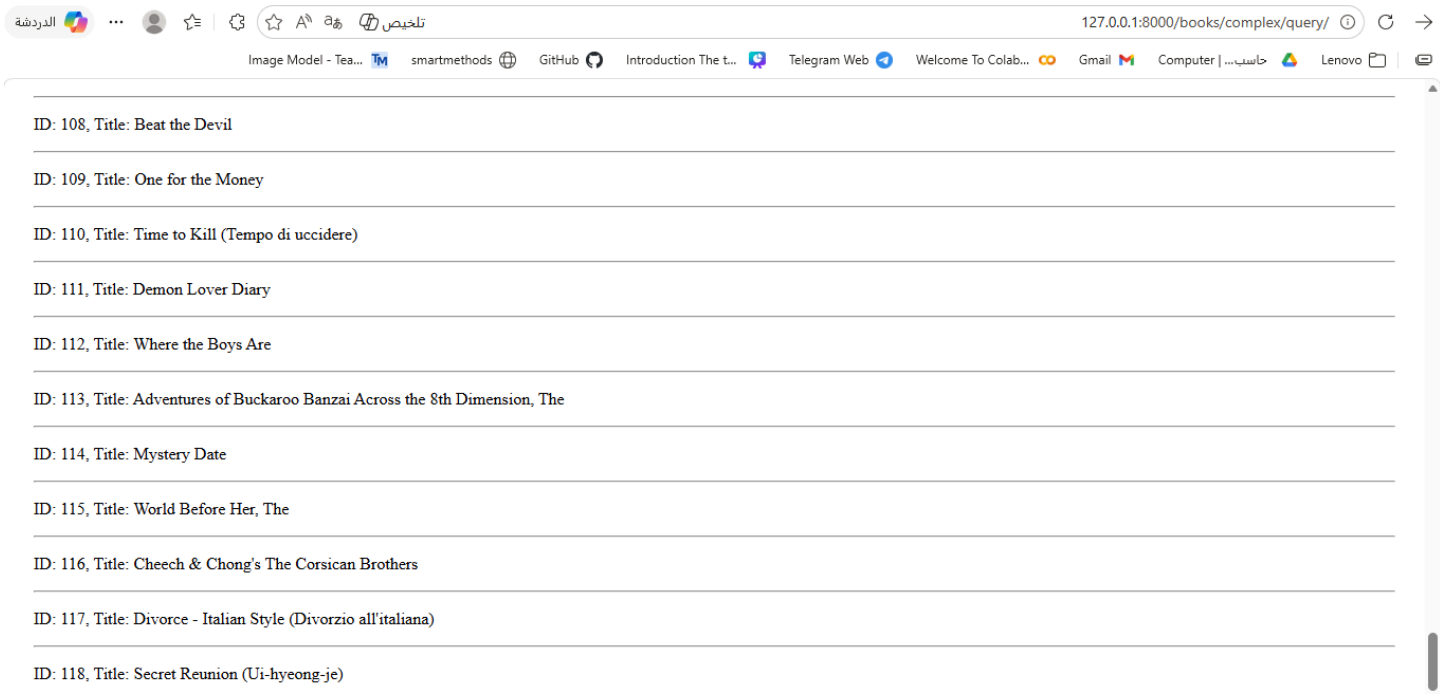


Step 3: Export the data to any file you want; I recommend SQL. Then, import that data into your database. You can use DB SQLite Browser, with the following steps:

- Open db.sqlite3
- Tab to execute SQL window

- Paste the content of SQL downloaded file into the windows
- Hit execute





Step 4: Also, to insert data from the Django from any view function, you can use the constructor function or create function:

- Use the constructor function

```
mybook = Book(title = 'Continuous Delivery', author = 'J.Humble and D. Farley', edition
= 1)
```

- Use the create function

```
mybook = Book.objects.create(title = 'Continuous Delivery', author = 'J.Humble and D.
Farley', edition = 1)
```

Either way, we must save the object once created:

```
mybook.save()
```

models.py 4 views.py 3

C: > Users > Lenovo > djangoenv > DjangoProjects > libraryproject > apps > bookmodule > views.py > ...

```
from django.shortcuts import render
from django.http import HttpResponse
from django.shortcuts import render
from .models import Book


def insert_data(request):
    obj1 = Book(title='Continuous Delivery', author='J.Humble and D. Farley', price=120.00, edition=3)
    obj1.save() # ضروري جداً لحفظ البيانات في القاعدة

    Book.objects.create(title='Reversing: Secrets of Reverse Engineer', author='E. Eilam', price=97.00, edi
    Book.objects.create(title='The Hundred-Page Machine Learning Book', author='Andriy Burkov', price=100.6

    return render(request, 'bookmodule/index.html')
```

```
models.py 4    views.py 3    urls.py 1 ×
C: > Users > Lenovo > djangoenv > DjangoProjects > libraryproject > apps > bookmodule > urls.py > ...

from django.urls import path
from . import views

urlpatterns = [
    path('search/', views.search_books, name='search_books'),
    path('', views.index, name="index"),
    path('list_books/', views.list_books, name="list_books"),
    path('one_book/', views.one_book, name="one_book"),
    path('aboutus/', views.aboutus, name="aboutus"),
    path('html5/links', views.lab5_links, name='lab5_links'),
    path('html5/text/formatting', views.lab5_formatting, name='lab5_formatting'),
    path('html5/listing', views.lab5_listing, name='lab5_listing'),
    path('html5/tables', views.lab5_tables, name='lab5_tables'),
    path('insert/', views.insert_data),
]
```

Task 3: Use simple queries using Django ORM.

Step 1: In 'apps/bookmodule/views.py', import book model:

```
from .models import Book
```

Step 2: Create URL '/books/simple/query' with a view function '`simple_query`' in 'apps/bookmodule/views.py':

```
def simple_query(request):
    mybooks=Book.objects.filter(title__icontains='and') # <- multiple objects
    return render(request, 'bookmodule/bookList.html', {'books':mybooks})
```

```
from .models import Book

def simple_query(request):
    mybooks = Book.objects.filter(title__icontains='and')
    return render(request, 'bookmodule/bookList.html', {'books': mybooks})
```

Step 3: In `'bookmodule/bookList.html'`, add Django tag to list books:

```
{% for obj in books %}
    ID: {{obj.id}}, Title: {{obj.title}}
    <hr>
{% endfor %}
```

Search

models.py 4 | views.py 3 | bookList.html | urls.py 1

C: > Users > Lenovo > djangoenv > DjangoProjects > libraryproject > apps > template > bookmodule > bo

```
1  {% extends "layouts/base.html" %}
2  {% block title %} Search Results {% endblock title %}
3
4  {% block content %}
5      <div style="margin: 20px;">
6          <h2>Search Results</h2>
7
8          {% if books %}
9              {% for obj in books %}
10                 <p>ID: {{ obj.id }}, Title: {{ obj.title }}</p>
11                 <hr>
12             {% endfor %}
13         {% else %}
14             <p>No books found matching your criteria.</p>
15         {% endif %}
16
17         <br>
18         <a href="{% url 'search_books' %}">Back to Search</a>
19     </div>
20 {% endblock content %}
```

Step 4: Now from the web browser, navigate to http://127.0.0.1:8000/books/simple/query

Home List Books About

Search Results

ID: 1, Title: Continuous Delivery

ID: 3, Title: The Hundred-Page Machine Learning Book

ID: 4, Title: Continuous Delivery

ID: 6, Title: The Hundred-Page Machine Learning Book

ID: 7, Title: Continuous Delivery

ID: 9, Title: The Hundred-Page Machine Learning Book

Back to Search

Task 4: Customize queries using common lookup conditions and keywords.

Step 1: Create URL '/books/complex/query' with a view function '`lookup_query`' in 'apps/bookmodule/views.py':

```
def complex_query(request):
    mybooks=books=Book.objects.filter(author__isnull =
False).filter(title__icontains='and').filter(edition__gte = 2).exclude(price__lte = 100)[:10]
    if len(mybooks)>=1:
        return render(request, 'bookmodule/bookList.html', {'books':mybooks})
    else:
        return render(request, 'bookmodule/index.html')
```

models.py 4 | views.py 3 | bookList.html | urls.py 1

C: > Users > Lenovo > djangoenv > DjangoProjects > libraryproject > apps > bookmodule > views.py > complex_query

```python
from django.shortcuts import render
from django.http import HttpResponse
from django.shortcuts import render
from .models import Book

from .models import Book

def complex_query(request):
    mybooks=books=Book.objects.filter(author__isnull = False).filter(title__icontains='and').filter(editior
    if len(mybooks)>=1:
        return render(request, 'bookmodule/bookList.html', {'books':mybooks})
    else:
        return render(request, 'bookmodule/index.html')

def insert_data(request):
```

models.py 4 | views.py 3 | bookList.html | urls.py 1

C: > Users > Lenovo > djangoenv > DjangoProjects > libraryproject > apps > bookmodule > urls.py > ...

```python

from django.urls import path
from . import views

urlpatterns = [
    path('search/', views.search_books, name='search_books'),
    path('', views.index, name="index"),
    path('list_books/', views.list_books, name="list_books"),
    path('one_book/', views.one_book, name="one_book"),
    path('aboutus/', views.aboutus, name="aboutus"),
    path('html5/links', views.lab5_links, name='lab5_links'),
    path('html5/text/formatting', views.lab5_formatting, name='lab5_formatting'),
    path('html5/listing', views.lab5_listing, name='lab5_listing'),
    path('html5/tables', views.lab5_tables, name='lab5_tables'),
    path('insert/', views.insert_data),
    path('simple/query/', views.simple_query, name='simple_query'),
    path('complex/query/', views.complex_query, name='complex_query'),
]
```

Step 2: Now from the web browser, navigate to http://127.0.0.1:8000/books/complex/query

Home List Books About

Search Results

ID: 1, Title: Continuous Delivery

ID: 2, Title: Reversing: Secrets of Reverse Engineer

ID: 3, Title: The Hundred-Page Machine Learning Book

ID: 4, Title: Continuous Delivery

ID: 5, Title: Reversing: Secrets of Reverse Engineer

ID: 6, Title: The Hundred-Page Machine Learning Book

ID: 7, Title: Continuous Delivery

ID: 8, Title: Reversing: Secrets of Reverse Engineer

ID: 9, Title: The Hundred-Page Machine Learning Book

Back to Search